

Sesión 17: Conectividad con **SQL Server**

Acceso Estructurado a Datos mediante C# y ADO.NET

IPC 2 | Capa de Persistencia | Facultad de Ingeniería USAC

De la Memoria al Disco



Integrando instrucciones SQL nativas dentro del flujo de ejecución del backend.

Objetivos Estructurados de la Sesión



Arquitectura ADO.NET

Comprender los componentes centrales de bajo nivel de .NET para el manejo de streams de datos.



Mitigación de Inyecciones

Eliminar de raíz vulnerabilidades críticas de manipulación SQL parametrizando comandos.



Patrón DAO

Encapsular el acceso a datos abstrayendo la lógica relacional fuera de los controladores MVC.

¿Qué es ADO.NET?

Es el conjunto de clases nativas incluidas en el framework de .NET diseñado para interactuar de forma homogénea con cualquier motor de base de datos relacional.

-  Comunicación directa de alto rendimiento.
-  Modelo desacoplado y orientado a proveedores.
-  Soporta arquitecturas conectadas y desconectadas.

Clase Principal	Responsabilidad Operativa
SqlConnection	Establece el canal físico hacia el servidor.
SqlCommand	Almacena e invoca la sentencia o script SQL.
SqlDataReader	Secuenciador de lectura rápida hacia adelante.

Anatomía de la Cadena de Conexión (Connection String)

Es el string de configuración que le indica al proveedor la ubicación exacta, el catálogo y las credenciales de seguridad del motor de datos:

```
// Ejemplo de Cadena de Conexión Estándar SQL Server
connectionString = "Server=localhost\\SQLEXPRESS;Database=ControlEstudiantes;Trusted_Connection=True;TrustServerCertificate=True;"
```

Parámetros clave:

Server: IP o nombre de la instancia. | Database: Catálogo relacional objetivo.

Trusted_Connection: Utiliza las credenciales de Windows (Seguridad Integrada).

Gestión Correcta de Recursos con el Bloque `using`

Las conexiones a bases de datos representan sockets físicos finitos. No liberarlas satura el pool de conexiones del servidor. El bloque using garantiza la destrucción implícita llamando a Dispose():

```
using (SqlConnection conexion = new SqlConnection(connectionString)) {  
    conexion.Open();  
    // El canal de red permanece abierto exclusivamente dentro de estas llaves  
}  
// Al salir, la conexión se cierra y libera automáticamente del pool, incluso si ocurre una excepción.
```

Ejecución de Comandos Escalares (ExecuteScalar)

Optimizado para consultas analíticas que devuelven **una única fila y una única columna** (un valor único como un COUNT, MAX, o SUM):

- ⚡ Alta eficiencia de memoria.
- ➡ Retorna un tipo object que requiere conversión explícita (Casting).

```
using (SqlConnection cmd = new SqlCommand(
    "SELECT COUNT(*) FROM Table", conexion)) {
    conexion.Open();
    int total = (int)cmd.ExecuteScalar();
}
```

Modificación de Registros mediante ExecuteNonQuery

Utilizado para operaciones de tipo **DML** (INSERT, UPDATE, DELETE) que alteran el estado de las tuplas pero no devuelven colecciones de registros.

Retorna un entero indicando las filas afectadas:

```
string query = "UPDATE Cursos SET Estado = 0 WHERECodigo = 'IPC2'";
using (SqlCommand cmd = new SqlCommand(query, conexion)) {
    conexion.Open();
    int filasAfectadas = cmd.ExecuteNonQuery();
    // Si filasAfectadas == 0, significa que ningún registro cumplió la condición del WHERE
}
```


Lectura Eficiente con SqlDataReader

El SqlDataReader mantiene una conexión activa por la cual fluyen los registros secuencialmente uno a uno. Se recorre mediante un ciclo evaluando el puntero con Read():

```
using (SqlDataReader reader = cmd.ExecuteReader()) {  
    while (reader.Read()) {  
        // Extracción indexada o fuertemente tipada por nombre de columna  
        int id = reader.GetInt32(0);  
        string nombre = reader["Nombre"].ToString();  
    }  
}
```

El Riesgo Crítico de la Inyección SQL

Concatenar directamente strings provistos por el usuario final permite alterar la lógica lógica del script SQL original, comprometiendo la base de datos completa:

```
// CÓDIGO ALTAMENTE VULNERABLE - NO REPETIR EN EL LABORATORIO
string query = "SELECT * FROM Usuarios WHERE Nick = '" + txtUser.Text + "' AND Pass = '" + txtPass.Text + "'";

// Si el atacante ingresa en txtUser: admin' --
// El motor interpreta: SELECT * FROM Usuarios WHERE Nick = 'admin' --' AND Pass = ''
// El símbolo '--' anula el resto de la consulta, evadiendo la autenticación por contraseña.
```

La Solución Definitiva: Consultas Parametrizadas

Los parámetros tratan los datos ingresados como valores literales invariables, imposibilitando que el motor de SQL compile código malicioso inyectado:

```
string query = "SELECT * FROM Usuarios WHERE Nick = @usuario AND Pass = @contrasenia";
using (SqlCommand cmd = new SqlCommand(query, conexion)) {
    // Vinculación explícita segura con tipos de datos definidos
    cmd.Parameters.Add(new SqlParameter("@usuario", System.Data.SqlDbType.VarChar) { Value = txtUser.Text });
    cmd.Parameters.AddWithValue("@contrasenia", txtPass.Text);
    // El framework escapa automáticamente caracteres especiales
}
```

Mapeo de Valores Nulos (DBNull.Value)

En SQL, la ausencia de valor se representa como NULL, la cual difiere del operador null nativo en C#.

Para enviar o leer campos que admiten nulos, se debe evaluar e interceptar mediante la clase base DBNull.Value:

```
// Envío seguro de un nulo hacia la base de datos
cmd.Parameters.AddWithValue("@Direccion",
string.IsNullOrEmpty(txtDir.Text) ? (object)DBNull.Value :
txtDir.Text);

// Lectura segura para evitar excepciones de casteo
int columnIndex = reader.GetOrdinal("Telefono");
string tel = reader.IsDBNull(columnIndex) ? "No asignado" :
reader.GetString(columnIndex);
```

Manejo de Errores con SqlException

Las fallas de conexión, bloqueos de concurrencia o violaciones de llaves primarias/foráneas lanzan excepciones de tipo `SqlException`. Capturarlas permite decodificar el error:

```
try {
    conexion.Open();
} catch (SqlException ex) {
    // ex.Number identifica el código exacto emitido por SQL Server
    if (ex.Number == 2627) { // Código estándar de violación de PK (Registro Duplicado)
        throw new Exception("El número de carné ingresado ya se encuentra registrado.");
    }
}
```

El Patrón DAO (Data Access Object)

Principio arquitectónico para aislar por completo la API de acceso a datos de la lógica de presentación del negocio. Los controladores nunca deben instanciar objetos Sql:

Capa de Software	Responsabilidad Operativa Directa
Controlador (MVC)	Recibe la solicitud HTTP, valida el ModelState y llama al DAO.
DAO / Repositorio	Abre la conexión, ejecuta los SqlCommand y mapea los registros a clases de C#.
Modelo / Entidad	Clases planas (POCO) que actúan como estructuras puras de datos.

Implementación de un DAO Completo (Lectura)

```
public class EstudianteDao {
    private readonly string _connString = "...";
    public List ObtenerTodos() {
        var lista = new List();
        using (var conn = new SqlConnection(_connString)) {
            using (var cmd = new SqlCommand("SELECT Id, Nombre FROM Estudiantes", conn)) {
                conn.Open();
                using (var reader = cmd.ExecuteReader()) {
                    while (reader.Read()) {
                        lista.Add(new Estudiante { Id = reader.GetInt32(0), Nombre = reader.GetString(1) });
                    }
                }
            }
        }
        return lista;
    }
}
```

Valor de la Unidad: Solidaridad en la Persistencia



Garantía de Datos

Protección Integral de la Información

Evitar la fuga de datos o fallas críticas mediante un desarrollo defensivo riguroso (parametrización, cierre de conexiones) es una muestra de ****solidaridad**** hacia la institución y los usuarios que confían su información en nuestros desarrollos.

"La solidez en la arquitectura de persistencia previene la pérdida catastrófica de transacciones, resguardando el activo más valioso de los usuarios."

— Fundamentos de Ingeniería de Software

Invocación de Procedimientos Almacenados (Stored Procedures)

Para ejecutar rutinas precompiladas directamente en el servidor SQL, modificamos la propiedad CommandType del objeto comando:

```
using (SqlCommand cmd = new SqlCommand("SP_InscribirEstudiante", conexion)) {  
    // Especificación explícita del tipo  
    cmd.CommandType = System.Data.CommandType.StoredProcedure;  
  
    // Agregamos los parámetros requeridos por la firma del SP  
    cmd.Parameters.AddWithValue("@Carne", nuevoCarne);  
    conexion.Open();  
    cmd.ExecuteNonQuery();  
}
```

Uso Avanzado de Parámetros de Salida (Output Parameters)

Permite recuperar valores calculados por un procedimiento almacenado sin necesidad de abrir un DataReader pesado (ej. llaves primarias autogeneradas):

```
SqlParameter paramSalida = new SqlParameter("@IdGenerado", System.Data.SqlDbType.Int) {  
    Direction = System.Data.ParameterDirection.Output  
};  
cmd.Parameters.Add(paramSalida);  
  
cmd.ExecuteNonQuery();  
// Recuperación del valor después de la ejecución  
int nuevoId = (int)paramSalida.Value;
```

Manejo de Transacciones Locales (SqlTransaction)

Garantiza el cumplimiento de la propiedad **Atomicidad** de ACID cuando múltiples comandos pertenecen a una misma unidad lógica indivisible:

```
SqlTransaction transaccion = conexion.BeginTransaction();
try {
    SqlCommand cmd1 = new SqlCommand("INSERT INTO Pagos...", conexion, transaccion);
    cmd1.ExecuteNonQuery();

    SqlCommand cmd2 = new SqlCommand("UPDATE Saldos...", conexion, transaccion);
    cmd2.ExecuteNonQuery();

    transaccion.Commit(); // Guarda los cambios de forma permanente
} catch {
    transaccion.Rollback(); // Revierte todo al estado inicial ante cualquier fallo
}
```

Arquitectura Desconectada: SqlDataAdapter y DataSet

Permite descargar colecciones de datos completas a la memoria RAM del servidor web, liberando la conexión física inmediatamente después de la descarga:

-  Llena estructuras locales DataTable.
-  Ideal para reportes estáticos extensos.

```
SqlDataAdapter adapter = new SqlDataAdapter(cmd);
DataTable tablaValores = new DataTable();

// Abre, descarga y cierra la conexión internamente
adapter.Fill(tablaValores);

// Los datos se recorren localmente en memoria sin conexión activa
```

Maapeo Seguro de Fechas (DateTime)

Las diferencias de formato regional (DD/MM/AAAA vs AAAA-MM-DD) entre servidores web y bases de datos causan fallas de conversión recurrentes. Pasar objetos de tipo `DateTime` nativos evita dependencias de strings:

```
string query = "INSERT INTO Asignaciones (Fecha) VALUES (@fechaAsig)";
SqlCommand cmd = new SqlCommand(query, conexion);

// Se envía el objeto estructurado, dejando que el driver resuelva el formateo binario
cmd.Parameters.Add(new SqlParameter("@fechaAsig", System.Data.SqlDbType.DateTime) {
    Value = DateTime.Now
});
```

Inyección de Dependencias del Connection String

Hardcodear credenciales compromete la seguridad y previene la portabilidad del software. Centralizamos la configuración dentro del archivo estándar appsettings.json:

```
// Archivo: appsettings.json
{
  "ConnectionStrings": {
    "CadenaProduccion": "Server=USAC_SERVER;Database=Control;Integrated Security=True;"
  }
}
```


Lecturas Asíncronas (ExecuteReaderAsync)

Para aplicaciones de alto tráfico, evitamos el bloqueo de hilos del servidor web delegando llamadas I/O de forma asíncrona mediante los operadores `async/await`:

```
using (var cmd = new SqlCommand("SELECT * FROM Cursos", conexion)) {  
    await conexion.OpenAsync();  
    using (var reader = await cmd.ExecuteReaderAsync()) {  
        while (await reader.ReadAsync()) {  
            // Procesa registros asíncronamente liberando CPU  
        }  
    }  
}
```

Vinculación Completa del Flujo: Formulario a DB

Flujo Completo de un Requerimiento Técnico:

1. **Vista:** El usuario ingresa la información del nuevo curso en inputs HTML vinculados.
2. **Controlador:** Valida el modelo y llama al método InsertarCurso(dto) del DAO.
3. **DAO:** Abre conexión, mapea los parámetros de C# a tipos SQL Server y confirma la transacción.

Buenas Prácticas Críticas en Persistencia

-  **NUNCA** concatenar variables directas en comandos de texto SQL.
-  **SIEMPRE** envolver objetos SqlConnection, SqlCommand y SqlDataReader en bloques using.
-  Centralizar los Connection Strings fuera del código duro de compilación.

Caso de Estudio Integrado para Laboratorio

Requerimiento: Sistema de Control de Notas con validación de cupos.

Flujo Operativo	Componente ADO.NET Utilizado
Verificar cupos disponibles en el salón.	ExecuteScalar (Retorna entero único)
Efectuar el cargo del pago e inscribir al alumno.	SqlTransaction (Asegura consistencia mutua)

Criterios de Evaluación de Conectividad

Defensa de Inyecciones

Penalización total si se encuentran concatenaciones directas de strings en consultas de bases de datos.

Liberación del Sockets

Verificación del uso correcto de bloques de clausura para evitar saturar el pool de hilos de red.

¿Preguntas de Conectividad?

Integración Nativa de SQL Server mediante el Proveedor SqlClient de .NET Core.

Área de Soporte de Cátedra: Laboratorio IPC 2

Siguiente Paso Técnico de la Unidad:

Mapeadores de Objetos Relacionales

(ORM)

Introducción a Entity Framework Core para automatizar consultas SQL.